Semana 07 — Controles automatizados con umbral y veredicto

Grupo 1 · rol Púrpura · ejemplo de trabajo

Prof. César Rodríguez | ELECTRONICA | 1 de noviembre de 2026

Curso	CYBERSEGURIDAD — Ciberseguridad Aplicada
Semana	07 de 10
Fase	3 — Defensa y hardening
Grupo	1
Rol	Púrpura
Entrega	GitHub, carpeta semana_07/
Corte	Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.

1 Tu rol en la práctica

Instrumenta la detección del mismo evento: activa el registro, define la alerta y mide lo que se habría visto, incluidos los falsos positivos y negativos.

- Instrumentar el evento, activando el registro que lo vería y definiendo la alerta con su umbral.
- Comparar lo que se registró con lo que se ejecutó, ítem por ítem del paso a paso.
- Reportar falsos positivos y negativos y proponer el ajuste concreto de la regla.

2 Qué aporta este rol al entregable

La regla o consulta de detección y la tabla de lo detectado frente a lo ejecutado.

3 Ejemplo paso a paso

El rol Púrpura convierte la revision en un control que falla solo. Cada herramienta tiene umbral, veredicto y una salida que se puede citar en el informe.

3.1 Ejecutar analisis estatico y fijar el umbral

El umbral se declara antes de mirar el resultado. Un hallazgo de severidad alta bloquea la entrega; los de severidad baja se registran y se deciden.

```
mkdir -p semana_07/evidencia
bandit -r semana_07/codigo -f json -o semana_07/evidencia/bandit.json
    || true
python3 - <<'PY' | tee semana_07/evidencia/bandit_resumen.txt
import json
datos = json.load(open('semana_07/evidencia/bandit.json'))
for hallazgo in datos.get('results', []):
    print(hallazgo['severity'], hallazgo['test_id'], hallazgo['filename
        '])
PY
```

3.2 Auditar dependencias y generar el inventario

El inventario de componentes permite saber que se usa y que version. Sin el, una vulnerabilidad publicada no se puede correlacionar con lo instalado.

```
pip-audit -r semana_07/requirements.txt \
  -f json -o semana_07/evidencia/pip_audit.json || true
python3 - <<'PY' | tee semana_07/evidencia/sbom.txt
import json
datos = json.load(open('semana_07/evidencia/pip_audit.json'))
for dep in datos.get('dependencies', []):
    print(dep['name'], dep['version'], len(dep.get('vulns', [])))
PY
```

3.3 Buscar secretos antes de que lleguen al historial

El escaneo mira el arbol y el historial. Un secreto retirado del arbol sigue estando en el commit anterior.

```
git grep -nEi '(api[_-]?key|secret|token|password)[[:space:]]*=' \
  > semana_07/evidencia/secretos.txt || true
git log --all -p -S 'BEGIN RSA PRIVATE KEY' --oneline \
  >> semana_07/evidencia/secretos.txt || true
```

3.4 Convertir el resultado en veredicto

El veredicto por herramienta es pasa, falla o no_verificado. Si una herramienta no se pudo ejecutar, el veredicto no es pasa.

3.5 Control, umbral y veredicto

Control, umbral y veredicto

Control	Umbral	Hallazgos	Veredicto	Accion
Bandit	Severidad alta igual a cero	Conteo del resumen	Pasa o falla	Corregir antes de entregar
pip-audit	Cero vulnerabilidades conocidas	Conteo del SBOM	Pasa o falla	Actualizar dependencia
Secretos	Cero coincidencias	Conteo del escaneo	Pasa o falla	Rotar y retirar
Regresion funcional	Todo en verde	Resultado de las pruebas	Pasa o falla	Corregir el parche

El veredicto se copia del resumen, no se interpreta de memoria. Un control no ejecutado se escribe como no verificado.

4 Qué guardar en el repositorio

Se entrega la salida cruda de los tres controles, el resumen con veredicto y la decision tomada por cada hallazgo.

```
semana_07/
|-- informe.md
|-- codigo/
|-- requirements.txt
|-- evidencia/
|   |-- bandit.json
|   |-- bandit_resumen.txt
|   |-- pip_audit.json
|   |-- sbom.txt
|   `-- secretos.txt
`-- README.md
```

- Los umbrales estan escritos antes de ejecutar los controles.
- El veredicto de cada herramienta se apoya en su resumen.
- Un control no ejecutado figura como no verificado.
- El escaneo de secretos incluye el historial.

5 Cómo se corrige

Criterio	Peso
Correcciones que eliminan la causa raíz, no el síntoma	3,0
Demostración antes y después con el mismo payload	2,5
Controles automatizados ejecutados e interpretados	2,0
Criterio de aceptación que previene la regresión	1,5
Sin regresiones funcionales en la aplicación	1,0
Total	**10,0**

- Un analisis estatico con cero hallazgos no demuestra que el codigo sea seguro.
- Las dependencias actualizadas sin prueba de regresion no son una correccion.

Recuerda

El trabajo es independiente: cada grupo usa su propio repositorio y su propio laboratorio, y la comparación de resultados es en clase. Corte: Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.